

Day 2 SQL scorecard — detailed ratings, corrections and Day 1 vs Day 2 comparison

DAY 2 RESULTS



Improving — window functions are getting sharp

Clear growth from Day 1. Q2, Q4, Q5 were strong. Main gaps: GROUP BY misuse in CTEs, running total PARTITION bug, and skipping Q7 string functions.

Full marks	Partial	Failed / skip	vs Day 1
3	5	2	+0.9

DAY 1 VS DAY 2

Day 1	6.0 / 10
Day 2	6.9 / 10

QUESTION-BY-QUESTION BREAKDOWN

Q1 — CASE WHEN ★★★★☆ 7/10

Good: CTE with CASE WHEN is the right approach. The labeling logic is correct. Bug: You wrote two separate SELECT statements — the second SELECT ... FROM label is outside the CTE scope and will error. Combine them into one query using the CTE for the count.

```
WITH labelled AS (  
  SELECT order_id, amount,  
    CASE WHEN amount > 1000 THEN 'High'  
      WHEN amount BETWEEN 500 AND 1000 THEN 'Medium'  
      ELSE 'Low'  
    END AS category  
  FROM orders  
)  
SELECT category, COUNT(*) AS no_of_orders  
FROM labelled  
GROUP BY category  
ORDER BY no_of_orders DESC;
```

Q2 — Self JOIN ★★★★★ 10/10

Perfect. Self-join with clean aliases, correct ON condition combining the join key and the salary filter. This is exactly how you'd write it in production. Nothing to fix.

Q3 — Running totals ★★★☆☆ 4/10

Key bug: PARTITION BY sale_date resets the running total for every date — you get the total per day, not a cumulative total across all days. For a true running total, remove PARTITION BY entirely (or keep it only if you want per-product running totals).

ORDER BY inside OVER() is what makes it cumulative.

```
SELECT sale_id, sale_date, amount,  
       SUM(amount) OVER (ORDER BY sale_date, sale_id) AS running_total  
FROM sales  
WHERE EXTRACT(YEAR FROM sale_date) = 2023  
ORDER BY sale_date, sale_id;
```

Q4 — Conditional aggregation (PIVOT)

★★★★☆ 8/10

Great technique — SUM(CASE WHEN QUARTER = N ...) is exactly right. One omission: Missing **GROUP BY category** and **category** in the SELECT. Without it you get a single grand-total row, not one row per category. Also add a WHERE for 2023.

```
SELECT category,  
       SUM(CASE WHEN EXTRACT(QUARTER FROM sale_date) = 1 THEN amount ELSE 0 END) AS q1,  
       SUM(CASE WHEN EXTRACT(QUARTER FROM sale_date) = 2 THEN amount ELSE 0 END) AS q2,  
       SUM(CASE WHEN EXTRACT(QUARTER FROM sale_date) = 3 THEN amount ELSE 0 END) AS q3,  
       SUM(CASE WHEN EXTRACT(QUARTER FROM sale_date) = 4 THEN amount ELSE 0 END) AS q4  
FROM sales  
WHERE EXTRACT(YEAR FROM sale_date) = 2023  
GROUP BY category  
ORDER BY category;
```

Q5 — LEAD & LAG

★★★★★ 10/10

Flawless. Correct PARTITION BY customer_id, correct ORDER BY order_date, LAG and LEAD used perfectly. First and last rows will naturally return NULL for prev/next — which is the correct behaviour.

Q6 — EXISTS vs IN

★★★★☆ 6/10

NOT EXISTS version is correct — great structure and correlated subquery logic. NOT IN version has two bugs: (1) **NOT IN 2024** is invalid — IN requires a list or subquery, not a scalar. (2) The JOIN approach doesn't correctly isolate "ordered in 2023 but not 2024" — it returns everyone who has any order not in 2024.

```
-- NOT IN (correct):  
SELECT customer_id, name FROM customers  
WHERE customer_id IN (  
    SELECT customer_id FROM orders  
    WHERE EXTRACT(YEAR FROM order_date) = 2023  
)  
AND customer_id NOT IN (  
    SELECT customer_id FROM orders  
    WHERE EXTRACT(YEAR FROM order_date) = 2024  
);  
  
-- NOT EXISTS (your version was correct — minor style fix):  
SELECT c.customer_id, c.name  
FROM customers c  
WHERE EXISTS (  
    SELECT 1 FROM orders o  
    WHERE o.customer_id = c.customer_id  
    AND EXTRACT(YEAR FROM o.order_date) = 2023
```

```

)
AND NOT EXISTS (
  SELECT 1 FROM orders o
  WHERE o.customer_id = c.customer_id
  AND EXTRACT(YEAR FROM o.order_date) = 2024
);

```

Q7 — String functions

☆☆☆☆☆ 0/10

Skipped. String functions are very common in DE pipeline work — data cleaning, email parsing, log processing. Make sure to attempt this next time. Here's the reference answer to study:

```

SELECT user_id, email,
  SPLIT_PART(email, '@', 1) AS username,
  SPLIT_PART(email, '@', 2) AS domain,
  INITCAP(REPLACE(SPLIT_PART(email, '@', 1), '.', ' ')) AS full_name
FROM users;

-- If SPLIT_PART not available (e.g. SQL Server):
-- SUBSTRING(email, 1, CHARINDEX('@', email) - 1) AS username
-- SUBSTRING(email, CHARINDEX('@', email) + 1, LEN(email)) AS domain

```

Q8 — Multi-level CTE

★★★★☆ 5/10

Right idea — DENSE_RANK in a CTE is the correct approach. Wrong final SELECT: You used GROUP BY 1 with a HAVING on a CASE expression — this is convoluted and incorrect. After ranking, simply filter WHERE d_rank = 2. No GROUP BY needed.

```

WITH ranked AS (
  SELECT name, department, salary,
    DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS d_rank
  FROM employees
)
SELECT department, name, salary
FROM ranked
WHERE d_rank = 2
ORDER BY department;

```

Q9 — Data quality check

★★★★☆ 5/10

Good instinct using a CTE and CASE for categorization. Two bugs: (1) You reference _rank inside the same SELECT's CASE — aliases aren't available within the same SELECT level. (2) CURRENT_DATE() has no parentheses in standard SQL. Better approach: use UNION ALL to produce a clear named report.

```

SELECT 'duplicate_order_ids' AS check_name,
  COUNT(*) AS violations
FROM (
  SELECT order_id FROM orders
  GROUP BY order_id HAVING COUNT(*) > 1
) t

UNION ALL

```

```

SELECT 'null_or_negative_amount',
       COUNT(*)
FROM orders
WHERE amount IS NULL OR amount < 0

UNION ALL

SELECT 'future_order_date',
       COUNT(*)
FROM orders
WHERE order_date > CURRENT_DATE;

```

Q10 — Gaps and islands

★★★★☆ 5/10

Core idea is right — date minus ROW_NUMBER is the classic gaps-and-islands pattern. Incomplete: You computed the group key but didn't finish — no MIN/MAX to get gap_start and gap_end, and GROUP BY gaps alone won't work without product_id. Also the PARTITION should not include stock_level in ROW_NUMBER since you already filtered for stock_level = 0.

```

WITH zero_stock AS (
  SELECT product_id, stock_date,
         stock_date - CAST(ROW_NUMBER() OVER (
           PARTITION BY product_id ORDER BY stock_date
         ) AS INT) AS grp
  FROM inventory
  WHERE stock_level = 0
)
SELECT product_id,
       MIN(stock_date) AS gap_start,
       MAX(stock_date) AS gap_end
FROM zero_stock
GROUP BY product_id, grp
ORDER BY product_id, gap_start;

```

What improved from Day 1

Window functions are now a real strength — Q2 and Q5 were flawless. You're also applying CTEs more naturally. The PIVOT technique in Q4 showed solid instinct even if category was missing. Your NOT EXISTS in Q6 was correctly structured — that's a real DE skill.

Focus for Day 3

1. String functions — SPLIT_PART, SUBSTRING, REPLACE, INITCAP. Don't skip these.
2. Running totals — no PARTITION BY when you want a global cumulative sum.
3. After ranking in a CTE, filter with WHERE rank = N — never use GROUP BY + HAVING for this.
4. Finish your hard queries — Q10 had the right skeleton, just needed MIN/MAX to complete it.
5. UNION ALL for multi-check reports — cleaner than CASE inside a window function.

Start Day 3 ↗